# CS5730 Operating Systems: 2022 Midterm Solutions

The 2022 paper only, in order. Each entry gives the year, the full question as printed, then a model answer sized to fit the space on the exam paper (prose kept to about 250 words; code, tables and diagrams added where the question needs them). Repeated questions use the same answer every time.

| Year | Paper | Questions |
|---|---|---|
| 2022 | CS5730 Operating Systems Midterm (2022/11/22) | 11 answers |

# 2022 · CS5730 Operating Systems Midterm (2022/11/22)

**Essay Questions: Your answers should be as concise and clear as possible**

**Q1** (5 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2022

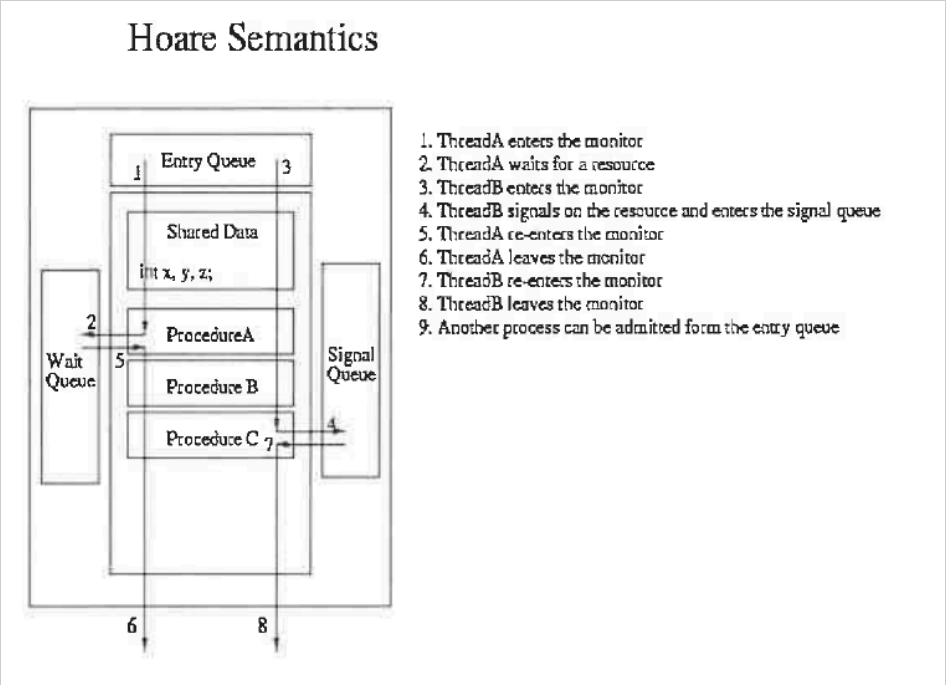
**ANSWER**

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

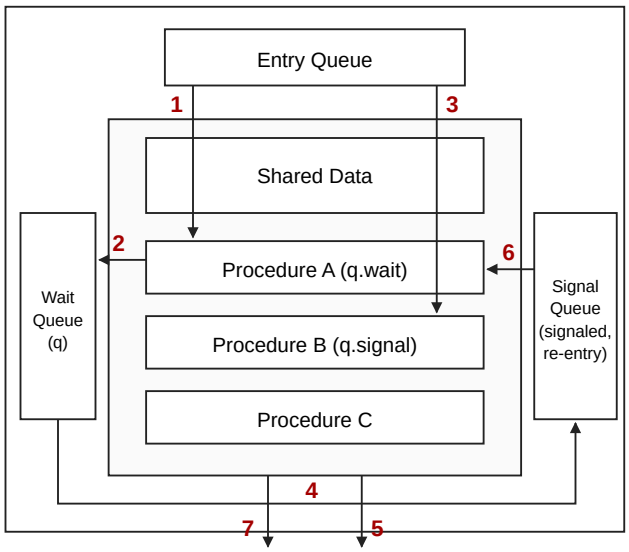
**Q2** (15 pt) Note that only one process is active within the monitor. Suppose `A` associated with the condition variable `q` is resumed when `B` invokes `q.signal()`. We discussed two possibile solutions in class. One is **signal and wait**, and the flow is illustrated in the following (Hoare semantics). Please modify the Hoare semantics and depict the other solution (i.e., **signal and continue**). 2022




**ANSWER**

**Signal-and-continue:** when B executes `q.signal()`, **B keeps running** in the monitor. The waiting process A is only moved from q's wait queue to a ready/re-entry queue (the "signal queue" now holds the *signaled*

process). A resumes after B leaves the monitor or waits.




1. Thread A enters the monitor
2. A waits on q → q's wait queue (monitor released)
3. Thread B enters the monitor
4. B signals q: A is moved from wait queue → signal queue; **B CONTINUES running**
5. B leaves the monitor
6. A re-enters the monitor (signal queue has priority over entry queue)
7. A leaves the monitor
8. Another process can be admitted from the entry queue

Difference from Hoare: signal queue now holds the SIGNALED process, not the signaler; 8 steps instead of 9.

Compared with Hoare: in Hoare's version A runs immediately and the signaler B waits in the signal queue. In signal-and-continue, A may find the condition changed by the time it runs, so waiters should re-check it with `while (!cond) q.wait();`. There are fewer context switches, and this is what Java and Pthreads use.

**Q3** (20 pt) In class, we have discussed implementing monitor with **signal and wait** via semaphores as follows. 2022

```
Variables
    semaphore mutex;    // (initially = 1)
    semaphore next;     // (initially = 0)
    int next-count = 0;

Each procedure F will be replaced by
    wait(mutex);
        ...
        // body of F;
        ...
    if (next_count > 0)
        signal(next)
    else
        signal(mutex);

For each condition variable x,
    semaphore x_sem;    // (initially = 0)
    int x-count = 0;

The operation x.wait can be implemented as:
    x-count++;
    if (next_count > 0)
        signal(next);
    else
        signal(mutex);
    wait(x_sem);
    x-count--;

The operation x.signal can be implemented as:
    if (x-count > 0) {
        next_count++;
        signal(x_sem);
        wait(next);
        next_count--;
    }
```

Please modify the above code to implement **signal and continue**.

**ANSWER**

In signal-and-wait the **signaler** blocks on `next`. For **signal-and-continue**, the signaler must **not** block. Instead, the **signaled** process waits on `next` to re-enter after the signaler leaves. Changed lines are marked NEW:

```
Variables
    semaphore mutex;      // (initially = 1)
    semaphore next;       // (initially = 0)
                          // = signaled procs
                          //   waiting to re-enter
    int next_count = 0;

Each procedure F (unchanged)
    wait(mutex);
        ...
        // body of F;
        ...
    if (next_count > 0)
        signal(next);     // signaled proc first
    else
        signal(mutex);
```

```
For each condition variable x
    semaphore x_sem;      // (initially = 0)
    int x_count = 0;

x.wait
    x_count++;
    if (next_count > 0)
        signal(next);
    else
        signal(mutex);
    wait(x_sem);          // sleep until signaled
    wait(next);           // NEW: wait until the
                          //   signaler leaves/waits
    next_count--;         // NEW: back in monitor
                          // (x_count-- moved to
                          //   x.signal)

x.signal                  // signal-and-continue
    if (x_count > 0) {
        x_count--;        // NEW: done by signaler
        next_count++;
        signal(x_sem);    // wake one waiter
        // REMOVED: wait(next); next_count--;
    }                     // signaler continues
```

Why: the signaler no longer suspends itself. The woken process must re-acquire the monitor, and it gets it through `next`, ahead of the entry queue, when the signaler exits or waits. x_count is decremented by the signaler, so two consecutive signals cannot wake the same single waiter twice. Callers should use `while (!B) x.wait();`.

**Q4** (10 pt) Suppose a architecture use 48-bit addressing with 4K paging on 64-processors. How much memory space a process may have? How many entries a page table may have? How should a page table be stored in memory? 2022

**ANSWER**

**Memory space per process:** 48-bit addresses give $2^{48}$ bytes = **256 TB** of logical address space.

**Page-table entries:** 4 KB = $2^{12}$ gives a 12-bit offset, so the page number is 48 − 12 = 36 bits. That makes **$2^{36}$ ≈ 68.7 billion (64 G) entries**. On a 64-bit processor each entry is 8 bytes, so a single flat table would need $2^{36} \times 8 = 2^{39}$ B = **512 GB per process**.

**How to store it:** a flat table is far too large to keep, let alone contiguously. **Page the page table** with a **multi-level (hierarchical) page table**. x86-64 uses **4 levels**:

| level 4: 9 bits | level 3: 9 bits | level 2: 9 bits | level 1: 9 bits | offset: 12 bits |
|---|---|---|---|---|

Each table at each level has $2^9$ = 512 entries × 8 B = 4 KB, exactly one page, so tables are paged themselves. **Only the tables for parts of the address space actually in use are allocated**, which keeps the memory cost small. Alternatives: a **hashed page table** (good for sparse 64-bit spaces) or an **inverted page table** (one entry per physical frame). A **TLB** hides the extra memory accesses.

## 5. Multilevel queue scheduling

**Q5.1** (5 pt) List two possible scheduling among queues, which we discussed in class. 2022

**ANSWER**

Two ways to schedule **among** the queues of a multilevel queue:

1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a

higher queue preempts a lower one. Lower queues may **starve**.

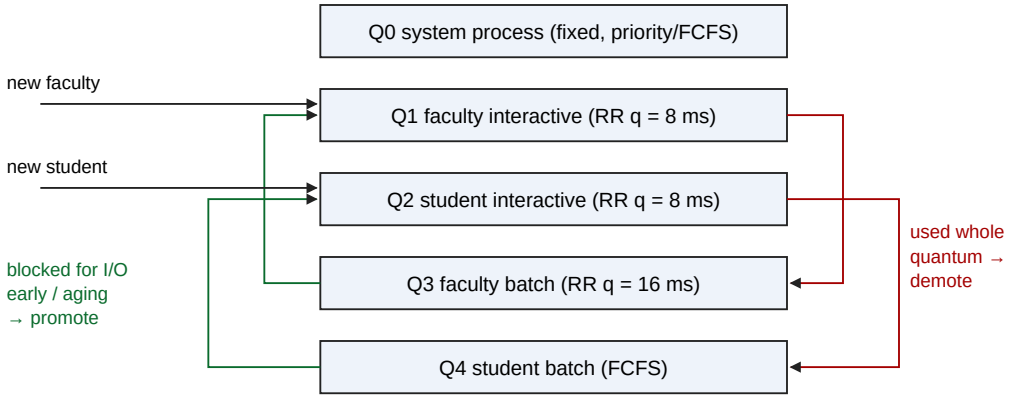
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

**Q5.2** (5 pt) Modify the following multilevel queue scheduling system such that a new process will automatically enter the appropriate queue after a few runs. 2022

| system process |
| --- |
| faculty interactive process |
| student interactive process |
| faculty batch process |
| student batch process |

**ANSWER**

Turn it into a **multilevel feedback queue**: a process may **move between queues** according to its observed behaviour, so it reaches the right queue automatically after a few runs.




Higher queue preempts lower (fixed priority between queues); Q0 never moves.

- A new process enters the **interactive** queue of its class (faculty → Q1, student → Q2), which uses a short quantum.
- If it **uses its whole quantum**, it is CPU-bound, so it is **demoted** to the batch queue of its class (Q1→Q3, Q2→Q4).
- If it **blocks for I/O before the quantum ends**, it is interactive, so it stays in or is **promoted** back to the interactive queue (Q3→Q1, Q4→Q2).
- **Aging** promotes processes that wait too long, preventing starvation. System processes stay fixed in Q0.

Defined by: number of queues, scheduling algorithm per queue, when to promote, when to demote, and which queue a new process enters.

## 6. Multilevel queue scheduling (heading as printed)

**Q6.1** (5 pt) What is **busy-waiting** in the discussion of critical section problem? Explain how **busy-waiting** may degrade the system performance. 2022

**ANSWER**

**Busy waiting** (spinning) means a process waiting to enter its critical section stays in a loop that keeps testing a condition until it can enter, e.g. `while (test_and_set(&lock));` or `while (S <= 0);`. It never gives up the CPU while waiting.

**Degraded performance:** the waiting process **wastes CPU cycles** doing no useful work, and other ready processes could have used them. On a single CPU, the spinner uses its entire time slice while the lock holder cannot run to release the lock, so everyone waits longer. If the spinner has higher priority than the lock holder, the holder may never run, and the system can even **deadlock** (priority inversion). Busy waiting is acceptable only for very short waits on multiprocessors, where it avoids a context switch.

**Q6.2** (5 pt) How to implement semaphore without **busy-waiting** using `block` and `wakeup`? You need to write down the definition of data structure `semaphore` and `wait` and `signal` operations. 2022

**ANSWER**

Each semaphore has an integer value and a **queue of waiting processes**. Instead of spinning, a process that must wait is **blocked** (moved to the waiting state and placed on the semaphore's queue). `signal` **wakes up** one waiting process (moves it to the ready queue).

```
typedef struct {
    int value;
    struct process *list;  /* queue of waiting PCBs */
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();               /* running -> waiting */
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);             /* waiting -> ready */
    }
}
```

If value is negative, |value| is the number of waiting processes. block() and wakeup() are system calls. wait() and signal() must still be atomic: disable interrupts inside them on a uniprocessor, or use a spinlock on a multiprocessor. That busy wait is limited to these few instructions, so it is short and rare.

**Q7** (10 pt) We have discussed the correct solution to the consumer-producer problem, which does not use any synchronization mechanism, as shown below. Please explain why it can only use `BUFFER_SIZE-1` elements of buffer. 2022

```
Producer:
    while (true) {
        /* produce an item in next produced */
        while (((in + 1) % BUFFER_SIZE) == out)
            ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
    }

Consumer:
    while (true) {
        while (in == out)
            ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        /* consume the item in next consumed */
    }
```

**ANSWER**

The solution uses only `in` (next free slot) and `out` (next full slot) to describe the buffer state, and it defines **empty** as `in == out` (consumer waits).

If the producer were allowed to fill **all** BUFFER_SIZE slots, then after the last insert `in` would wrap around and also equal `out`. **"Full" would look exactly like "empty"**, and the consumer could not tell them apart. To avoid this ambiguity, the producer treats the buffer as **full when** `(in + 1) % BUFFER_SIZE == out`, so one slot is always left unused.

Example with BUFFER_SIZE = 10: out = 0 and the producer has filled b0–b8, so in = 9. Now (9+1) % 10 = 0 = out, and the producer waits with only **9 items** stored. b9 stays empty; filling it would make in = 0 = out, which means "empty".

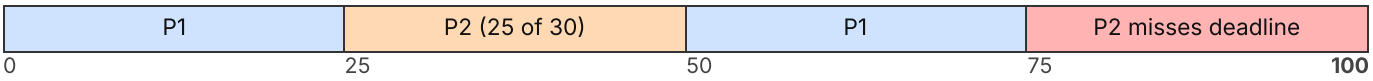
To use all N slots, add a `counter` of items (which then needs synchronization because counter++/counter-- race) or a full/empty flag.

**8. Consider two processes, $P_1$ and $P_2$, where $p_1 = d_1 = 50$, $t_1 = 25$, $p_2 = d_2 = 75$, and $t_2 = 30$.**

**Q8.1** (10 pt) Can these two processes be scheduled using rate-monotonic scheduling (RMS)? Illustrate your answer using a Gantt chart such as the ones we did in class. 2022

**ANSWER**

CPU utilization U = 25/50 + 30/75 = 0.50 + 0.40 = **0.90**. The RMS guaranteed bound for N = 2 is $2(2^{1/2} - 1)$ ≈ **0.83**. Since U > 0.83, schedulability is not guaranteed, so draw the chart. RMS uses static priority by period: P1 (period 50) has **higher priority** than P2 (period 75).




- 0–25: P1 runs (deadline 50 met).
- 25–50: P2 runs, but at t = 50 P1's second period starts and, having higher priority, **preempts P2**. P2 has done only 25 of its 30 ms.
- 50–75: P1 runs (deadline 100 met).
- t = 75: P2's deadline arrives with 5 ms still left, so **P2 misses its deadline**.

**No, P1 and P2 cannot be scheduled with RMS.**

**Q8.2** (10 pt) Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling. 2022

**ANSWER**

EDF assigns priority dynamically: the job with the **earliest absolute deadline** runs (on a tie, the running job continues). U = 0.90 ≤ 1, so EDF can schedule them.




- t=0: P1 (deadline 50) vs P2 (deadline 75), so P1 runs 0–25. Then P2 starts at 25.
- t=50: P1's 2nd job arrives (deadline 100). P2's deadline 75 is earlier, so **P2 is not preempted** and finishes at 55 (deadline 75 met). P1 runs from 55.
- t=75: P2's 2nd job arrives (deadline 150). P1's deadline 100 is earlier, so P1 finishes at 80 (met). P2 runs 80–110.
- t=100: P1's 3rd job (deadline 150) ties with P2, so no preemption. P2 finishes at 110 (met), and P1 runs 110–135 (met). Idle 135–150.
- At t = 150 (LCM of 50 and 75) both tasks restart, so the pattern repeats.

**All deadlines are met under EDF.**